

# 4-bit MIPS — Control Signal Tables

Instruction sequence **IACKOLEMPBJDNFHG** | Signal values decoded directly from the Control ROM

## Master Table — All 16 Opcodes

| Instruction | Opcode (dec) | Opcode (bin) | Format     | RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|-------------|--------------|--------------|------------|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| <b>sll</b>  | 0            | 0000         | S          | 0      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 101   | 1        | 0       |
| <b>add</b>  | 1            | 0001         | R          | 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 000   | 0        | 0       |
| <b>sub</b>  | 2            | 0010         | R          | 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 001   | 0        | 0       |
| <b>nor</b>  | 3            | 0011         | R          | 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 100   | 0        | 0       |
| <b>bneq</b> | 4            | 0100         | I (branch) | X      | 0      | X         | 0         | 0        | 0         | 0      | 1          | 0    | 001   | 0        | 0       |
| <b>lw</b>   | 5            | 0101         | I (mem)    | 0      | 1      | 1         | 1         | 1        | 0         | 0      | 0          | 0    | 000   | 0        | 0       |
| <b>and</b>  | 6            | 0110         | R          | 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 010   | 0        | 0       |
| <b>sw</b>   | 7            | 0111         | I (mem)    | X      | 1      | X         | 0         | 0        | 1         | 0      | 0          | 0    | 000   | 0        | 0       |
| <b>j</b>    | 8            | 1000         | J          | X      | X      | X         | 0         | 0        | 0         | 0      | 0          | 1    | X     | 0        | 0       |
| <b>addi</b> | 9            | 1001         | I          | 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 000   | 0        | 0       |
| <b>srl</b>  | 10           | 1010         | S          | 0      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 110   | 1        | 0       |
| <b>subi</b> | 11           | 1011         | I          | 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 001   | 0        | 1       |
| <b>beq</b>  | 12           | 1100         | I (branch) | X      | 0      | X         | 0         | 0        | 0         | 1      | 0          | 0    | 001   | 0        | 0       |
| <b>andi</b> | 13           | 1101         | I          | 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 010   | 0        | 0       |
| <b>ori</b>  | 14           | 1110         | I          | 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 011   | 0        | 0       |
| <b>or</b>   | 15           | 1111         | R          | 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 011   | 0        | 0       |

**X** = don't care (the signal has no effect for that instruction, because nothing downstream consumes it). **ALUOp** encoding: 000 = ADD, 001 = SUB, 010 = AND, 011 = OR, 100 = NOR, 101 = shift left, 110 = shift right.

## Per-Opcode Detail

**sll — opcode 0 (0000) — S-format** — `sll $d, $s1, shamt`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 0      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 101   | 1        | 0       |

S-type. IsShift=1 routes the shamt field to the ALU. RegDst=0 so the destination comes from instr[7:4], NOT the shift-amount field. | ALU operation: **Shift left** (101)

**add — opcode 1 (0001) — R-format** — `add $d, $s1, $s2`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 000   | 0        | 0       |

Standard R-type. RegDst=1 selects instr[3:0] as the destination register. | ALU operation: **ADD** (000)

**sub — opcode 2 (0010) — R-format** — `sub $d, $s1, $s2`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 001   | 0        | 0       |

Standard R-type. ALUOp=001 performs subtraction. | ALU operation: **SUB** (001)

**nor — opcode 3 (0011) — R-format** — `nor $d, $s1, $s2`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 100   | 0        | 0       |

Standard R-type bitwise NOR. | ALU operation: **NOR** (100)

**bneq — opcode 4 (0100) — I (branch)-format** — `bneq $s1, $s2, off`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|

|   |   |   |   |   |   |   |   |   |   |     |   |   |
|---|---|---|---|---|---|---|---|---|---|-----|---|---|
| X | 0 | X | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 001 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|-----|---|---|

BranchNot=1. Also subtracts; the branch is taken when Zero is LOW. | ALU operation: SUB (001)

|                                                             |              |
|-------------------------------------------------------------|--------------|
| RegDst → Shift, branch, imm value, lw, sw <del>lw, sw</del> | <u>ALUop</u> |
| ALUSrc → Offset <del>lw, sw</del> (lw, sw, imm value)       | 000 → ADD    |
| MemtoReg → lw                                               | 001 → SUB    |
| RegWrite → branch, sw <del>lw, sw</del>                     | 010 → AND    |
| MemRead → lw                                                | 011 → OR     |
| Memwrite → sw                                               | 100 → NOR    |
| Branch → beq                                                | 101 → SLL    |
| Branch not → bne                                            | 110 → SRL    |
| Jump → j                                                    |              |
| isShift → sll, srl                                          |              |
| issubi → subi                                               |              |

**lw — opcode 5 (0101) — I(mem)-format** — lw \$rt, off(\$base)

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 0      | 1      | 1         | 1         | 1        | 0         | 0      | 0          | 0    | 000   | 0        | 0       |

MemRead=1 and MemToReg=1 so the value written back comes from data memory rather than the ALU. ALUSrc=1 so the ALU computes base + offset. I ALU operation: **ADD** (000)

**and — opcode 6 (0110) — R-format** — and \$d, \$s1, \$s2

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 010   | 0        | 0       |

Standard R-type bitwise AND. I ALU operation: **AND** (010)

**sw — opcode 7 (0111) — I(mem)-format** — sw \$rt, off(\$base)

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| X      | 1      | X         | 0         | 0        | 1         | 0      | 0          | 0    | 000   | 0        | 0       |

MemWrite=1. RegWrite=0 (nothing is written back to the register file), so RegDst and MemToReg are don't-cares. I ALU operation: **ADD** (000)

**j — opcode 8 (1000) — J-format** — j target

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| X      | X      | X         | 0         | 0        | 0         | 0      | 0          | 1    | X     | 0        | 0       |

Jump=1. No register or memory activity at all, so nearly every other control signal is a don't-care.

**addi — opcode 9 (1001) — I-format** — addi \$d, \$s1, imm

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 000   | 0        | 0       |

I-type: ALUSrc=1 routes the immediate into the ALU; RegDst=0 selects instr[7:4]. I ALU operation: **ADD** (000)

**srl — opcode 10 (1010) — S-format** — `srl $d, $s1, shamt`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 0      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 110   | 1        | 0       |

S-type. IsShift=1. RegDst=0 for the same reason as sll. | ALU operation: **Shift right** (110)

**subi — opcode 11 (1011) — I-format** — `subi $d, $s1, imm`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 001   | 0        | 1       |

I-type subtract. IsSubi=1 is asserted for this instruction. | ALU operation: **SUB** (001)

**beq — opcode 12 (1100) — I (branch)-format** — `beq $s1, $s2, off`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| X      | 0      | X         | 0         | 0        | 0         | 1      | 0          | 0    | 001   | 0        | 0       |

Branch=1. ALUOp=001 (subtract) so the ALU Zero flag indicates equality. RegWrite=0, so RegDst/MemToReg are don't-cares. | ALU operation: **SUB** (001)

**andi — opcode 13 (1101) — I-format** — `andi $d, $s1, imm`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 010   | 0        | 0       |

I-type bitwise AND with the immediate. | ALU operation: **AND** (010)

**ori — opcode 14 (1110) — I-format** — `ori $d, $s1, imm`

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 0      | 1      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 011   | 0        | 0       |

I-type bitwise OR with the immediate. RegDst=0, ALUSrc=1. | ALU operation: **OR** (011)

**or** — opcode 15 (1111) — R-format — or \$d, \$s1, \$s2

| RegDst | ALUSrc | Memto-Reg | Reg-Write | Mem-Read | Mem-Write | Branch | Branch Not | Jump | ALUOp | Is Shift | Is Subi |
|--------|--------|-----------|-----------|----------|-----------|--------|------------|------|-------|----------|---------|
| 1      | 0      | 0         | 1         | 0        | 0         | 0      | 0          | 0    | 011   | 0        | 0       |

Standard R-type bitwise OR. RegDst=1, ALUSrc=0 (both operands from registers). | ALU operation: **OR** (011)